

Lab Assignment: Student Performance Analyzer

Submission Details

- **Input Data:** The input.csv file that has been already provided.
- **Submission Platform:** Google Classroom
- **Submit:**
 - Your Java file (`StudentAnalyzer.java`)
 - Output CSV file produced
 - 3 screenshots demonstrating different outputs (e.g., top and bottom performers)

Objective

The goal of this assignment is to design a complete Java program that reads student exam records from a CSV file and writes performance results to an output file. The program will:

- Read file paths via `String[] args`
- Parse student data into objects
- Compute and store performance metrics
- Interact with the user to display top/bottom performers

Input Format

The input CSV file should be used. It follows this structure:

```
1 ID,Name,Midterm,Final
2 101,Alice,70,85
3 102,Bob,50,60
4 103,Charlie,90,95
5 ...
```

Program Requirements

General Features

- Accept input and output file paths as command-line arguments:

```
1 java StudentAnalyzer input.csv output.csv
```

- Read and parse each row of the CSV file into a `Student` object.
- For each student, calculate:
 - $\text{average} = 0.4 * \text{midterm} + 0.6 * \text{finalExam}$

– `status = PASS` if average ≥ 60 , otherwise `FAIL`

- Write an output file in the format:

```
1 ID,Name,Average,Status
2 101,Alice,79.0,PASS
3 102,Bob,56.0,FAIL
4 ...
```

- Console output must include:
 - Total students processed
 - Number of students passed
 - Top performer name and score

Top/Bottom Performer Interaction

After writing the output file, the program must prompt:

Would you like to see the best performing or worst performing students?

- If the user responds `yes` (case-insensitive), then ask:

How many of those would you like to see? You may also provide a percentage (e.g., 25%).

- User inputs either:
 - A number (e.g., 3)
 - A percentage string (e.g., 25%)

- Ask for direction:

Show worst or best? (Type 'worst' or 'best')

- Display sorted results accordingly on the console.

Class Design

You must implement a class **Student** with the following structure:

```
1 public class Student {
2     private int id;
3     private String name;
4     private double midterm;
5     private double finalExam;
6
7     public Student(int id, String name, double midterm, double
      ↪ finalExam) { ... }
8
9     public double computeAverage() { ... }
10
11    public String determineStatus() { ... }
12
13    public String getName() { ... }
14
15    public int getId() { ... }
16
17    // Additional getters and toString if necessary
18 }
```

Technical Requirements

- Use at least **6 methods**, each with clearly defined responsibilities:
 - **readStudentsFromCSV(String path)** **Reads** student data from a file and returns a list of Student objects.
 - **writeResultsToCSV(String path, List<Student> list)** **Writes** performance summaries to a new output file.
 - **computeAverage()** **Returns** the average score for a student.
 - **determineStatus()** **Returns** “PASS” or “FAIL” based on the average.
 - **promptPerformanceSummary(List<Student> list)** **Asks** the user if they want to see top/bottom students and prints the appropriate data.
 - **printSummary(List<Student> list)** **Prints** statistics: total, pass count, and top performer.
- Validate the number of command-line arguments.
- Use **ArrayList**, **Collections.sort**, or comparators as needed.
- Apply exception handling to manage missing files or parsing errors.